

Algorithmes et Pensée Computationnelle

Algorithmes de graphes

Le but de cette séance est de comprendre le fonctionnement des graphes et d'appliquer des algorithmes courants sur des graphes simples.

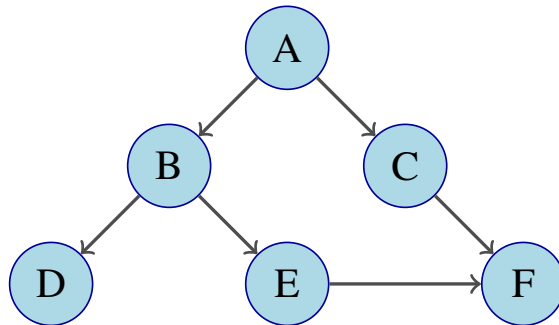
Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier `code`. Le temps indiqué (🕒) est à titre indicatif.

1 Parcours en largeur — Breadth-First Search

Question 1: (🕒 10 minutes) Adjacency list et adjacency matrix : Python

L'adjacency list (ou liste de contiguïté) et l'adjacency matrix (ou matrice de contiguïté) sont les deux méthodes dont nous disposons pour représenter un graphe.

Utilisez ces deux méthodes de représentations pour stocker le graphe ci-dessous dans un programme Python:



💡 Conseil

Pour l'adjacency list, utilisez un dictionnaire Python. Pour l'adjacency matrix, utilisez une liste multidimensionnelle (ou liste contenant une ou plusieurs listes).

>_ Solution

```
1 #Question 1 solution
2
3 adjacency_list_graph = {
4     'A' : ['B', 'C'],
5     'B' : ['D', 'E'],
6     'C' : ['F'],
7     'D' : [],
8     'E' : ['F'],
9     'F' : []
10 }
```

>_ Solution

```
12 adjacency_matrix_graphe = [[0,1,1,0,0,0],
13                             [0,0,0,1,1,0],
14                             [0,0,0,0,0,1],
15                             [0,0,0,0,0,0],
16                             [0,0,0,0,0,1],
17                             [0,0,0,0,0,0]]
```

Le fait que le graphe soit dirigé joue un rôle important pour la construction de ces représentations. Par exemple, pour l'adjacency list, B apparaît dans la liste correspondant à la clé A mais l'inverse n'est pas vrai. Cela signifie que l'on peut aller du sommet A au sommet B mais pas du sommet B au sommet A.

Question 2: (🕒 20 minutes) Breadth-First Search algorithm : Python

Nous allons maintenant nous intéresser au premier algorithme portant sur les graphes : **Breadth-First Search**. Le but du Breadth-First Search est de trouver tous les sommets atteignables à partir d'un sommet de départ.

Implémentez l'algorithme en suivant les étapes suivantes :

1. Initialisez :
 - une `file` (queue FIFO — de type liste) qui contient initialement le sommet de départ.
 - une liste, appelée `visited`, de sommets visités qui contient initialement le sommet de départ.
 - une liste, appelée `order`, qui stocke l'ordre d'enlèvement de la `file` — sommets proches au sommet de départ au début de la liste.
2. Tant que la `file` n'est pas vide, répétez les spécifications suivantes :
 - (a) Retirez le premier sommet de la `file` (utilisez `pop`) et appelez-le `u`.
 - (b) Ajoutez `u` à `order`
 - (c) Pour chaque sommet adjacent `v` de `u`: si `v` n'est pas dans `visited`, alors :
 - Insérez `v` à la fin de la `file`.
 - Insérez `v` à la fin de `visited`.
3. Quand la `file` devient vide, tous les sommets atteignables depuis le sommet de départ ont été visités en largeur.
4. Retournez `order`.

💡 Conseil

Quelques conseils pour l'implémentation de votre algorithme :

1. Utilisez un adjacency list comme définie précédemment à la question 1.
2. Pour le point (2), utilisez une boucle `while` avec la condition appropriée.
3. Pour parcourir les sommets adjacents, utilisez une boucle `for`.
4. L'algorithme devrait retourner une liste contenant l'ensemble des sommets atteignables.
5. Vous pouvez utiliser l'image du graphe pour déterminer si l'output de votre algorithme est correct.
6. Vous pouvez utiliser la méthode `pop()` pour supprimer un élément d'une liste et retourner l'élément supprimé. Cet élément peut être stocké dans une variable.

Référez-vous au pseudo-code présenté en cours.

>_ Solution

```
1 # Question 2
2
3 adjacency_list_graph = {
4     'A' : ['B', 'C'],
5     'B' : ['D', 'E'],
6     'C' : ['F'],
7     'D' : [],
8     'E' : ['F'],
9     'F' : []
10 }
11
12 def bfs(graph, start):
13     visited = list(start) # liste des sommets visités
14     queue = [start] # liste des sommets *dont les voisins* sont à
        visiter
15     order = [] # pour stocker l'ordre de sommets (sommets proches
        d'abord puis les sommets qui sont plus loin)
16     while len(queue) > 0: # tant que la queue n'est pas vide
17         u = queue.pop(0) # on stocke le premier élément de la queue,
            puis on l'enlève de la queue
18         order.append(u)
19         neighbors = graph[u] # on récupère la liste des sommets
            adjacents au sommet courant
20         for v in neighbors: # Puis on parcourt la liste de ceux-ci
21             if v not in visited:
22                 queue.append(v) # on les ajoute à la queue
23                 visited.append(v)
24     return order # on retourne la liste des sommets visités
        (=atteignables)
25
26
27 # Vérifions que l'algorithme fonctionne correctement
28 print(bfs(adjacency_list_graph, 'B'))
29 print(bfs(adjacency_list_graph, 'A'))
```

Si vous avez correctement écrit votre algorithme, l'output de celui-ci avec comme input notre graphe et le sommet B devrait être ['B', 'D', 'E', 'F'] et ['A', 'B', 'C', 'D', 'E', 'F'] pour B.

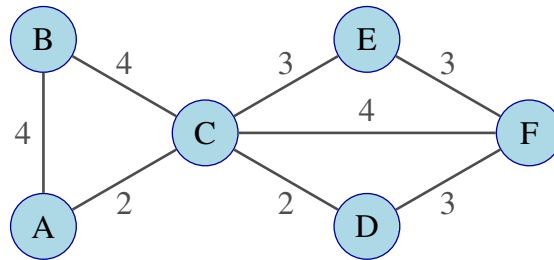
2 Arbre couvrant minimum (ACM) — Minimum Spanning tree (MST)

Nous allons maintenant nous intéresser à l'**algorithme de Kruskal**. Ce dernier s'applique uniquement aux **weighted graphs** (ou graphes pondérés). Ces derniers sont des graphes où les arêtes ont des poids, représentant par exemple une distance. L'algorithme de Kruskal a pour but de trouver un **arbre couvrant minimum** (ACM). Un graphe connexe est un graphe dans lequel il existe au moins un chemin entre n'importe quelle paire de sommets. Un arbre couvrant minimum S de G est un sous-graphe connexe de G tel que :

1. $V' = V$, c'est-à-dire que tous les sommets de G sont aussi dans S
2. (V', E') ne contient pas de cycle (pas de cycle dans S)
3. S est le graphe satisfaisant (1) et (2) et ayant la plus petite somme des poids
4. Pour chaque paire de sommets dans G, S contient le chemin *minimax* entre ces deux sommets. Le chemin *minimax* est un chemin pour lequel le poids de l'arête la plus lourde est minimisé.

Question 3: (🕒 5 minutes) Algorithme de Kruskal: Papier

Appliquez l'algorithme de Kruskal au graphe suivant:



💡 Conseil

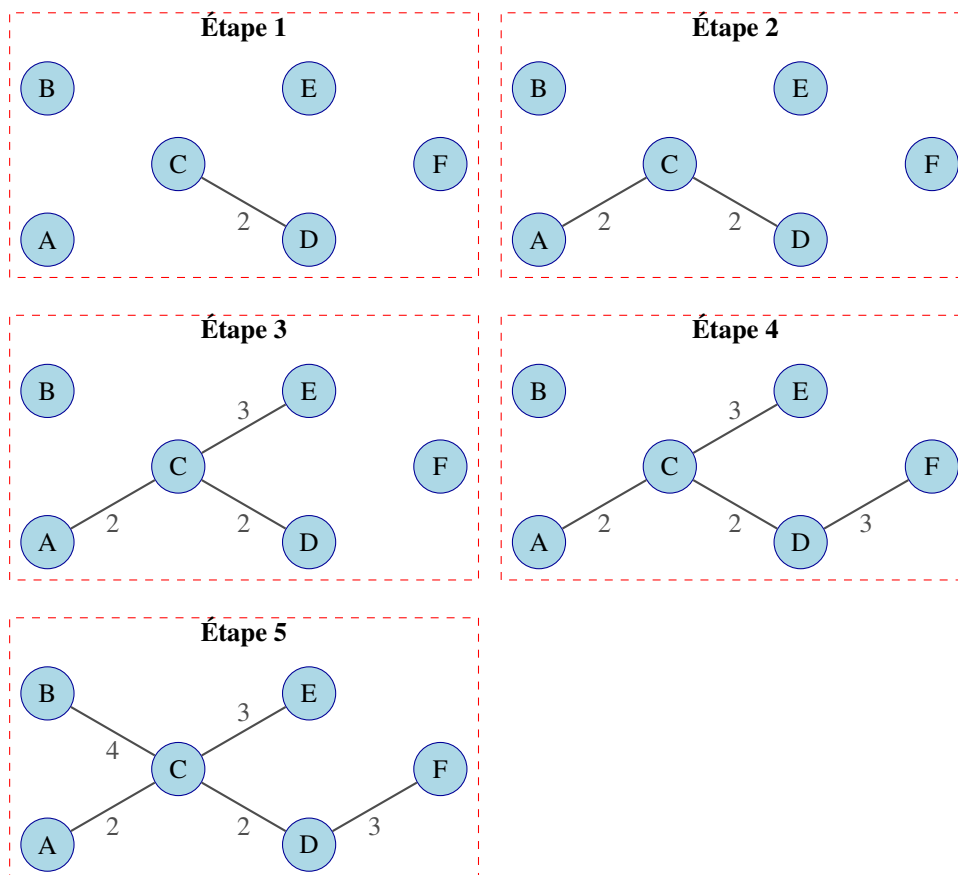
L'algorithme de Kruskal fonctionne de la façon suivante :

1. Classer les arêtes par ordre croissant de poids.
2. Prendre l'arête avec le poids le plus faible et l'ajouter à l'arbre (si 2 arêtes ont le même poids, choisir arbitrairement une des 2).
3. Vérifiez que l'arête ajoutée ne crée pas de cycle, si c'est le cas, supprimez-la.
4. Répétez les étapes (2) et (3) jusqu'à ce que tous les sommets aient été atteints.

Un arbre couvrant minimum (minimum spanning tree), s'il existe, a toujours un nombre d'arêtes égal au nombre de sommets moins un. Par exemple, ici notre graphe a 6 sommets. L'algorithme devrait donc s'arrêter lorsque 5 arêtes ont été choisies.

>_ Solution

Vous trouverez ci-dessous les étapes de la construction de l'ACM :



>_ Solution

L'algorithme s'arrête car tous les sommets ont été atteints. On voit bien que seules 5 arêtes ont été nécessaires. On remarque que l'ACM n'est pas unique dans ce cas comme on aurait pu choisir l'arête verticale tout à gauche du poids 4 au lieu de l'autre arête du poids 4.

Question 4: (🕒 30 minutes) Lecture du code — Algorithme de Kruskal: Python

Vous trouverez ci-dessous l'algorithme de Kruskal implémenté en Python (disponible sur Moodle). Lisez le code afin de vous assurer que vous avez bien compris le fonctionnement:

```
1 # Question 4
2
3 class Graph:
4     def __init__(self, vertices): # permet de créer un graphe lorsqu'on écrit
5         # p.ex. Graph(6), il faut notamment indiquer le nombre de sommets
6         self.V = vertices # le nombre de sommets
7         self.edges = [] # liste de tuples (u, v, w) ou w est le poids de
8         # l'arête entre u et v
9         self.parent = list(range(vertices)) # une liste [0, .., vertices - 1]
10        self.rank = [0]*vertices # une liste [0, ..,0] de longueur vertices
11
12    def add_edge(self, u, v, w): # ajoute une arête entre le sommet u et v
13        # avec un poids w
14        self.edges.append((u, v, w))
15
16    def find(self, i): # Correspond à la fonction Find-set(x) du cours
17        if self.parent[i] == i:
18            return i
19        self.parent[i] = self.find(self.parent[i])
20        return self.parent[i]
21
22    def union(self, x, y): # Correspond à la fonction Union(x,y) du cours
23        x_root = self.find(x)
24        y_root = self.find(y)
25
26        if x_root == y_root:
27            return False # x et y appartiennent déjà au même ensemble, on ne
28            # peut pas fusionner leurs ensembles
29
30        if self.rank[x_root] < self.rank[y_root]:
31            self.parent[x_root] = y_root
32        elif self.rank[x_root] > self.rank[y_root]:
33            self.parent[y_root] = x_root
34        else:
35            self.parent[y_root] = x_root
36            self.rank[x_root] += 1
37
38        return True # x et y n'appartiennent pas au même ensemble, on peut
39        # fusionner leurs ensembles
40
41    def kruskal(g : Graph):
42        result = []
43        edges = sorted(g.edges, key=lambda item: item[2]) # trie les arêtes
44        # par poids croissant
45        i = 0 # l'étape de l'itération
46        e = 0 # nombre d'arêtes pendant la construction de l'ACM
47
48        # Tant que le nombre d'arêtes est inférieur à V-1, notre sous-graphe
49        # n'atteint pas tous les sommets -> on continue
50        while e < g.V - 1:
51            u, v, w = g.edges[i] # self.graph contient les arêtes par ordre
```

```

croissant de poids, on commence avec i = 0
45     i = i + 1 # à l'itération suivante on voudra avoir la 2ème arête
    la plus légère, donc on incrémente
46
47     if g.union(u, v): # Si u et v font déjà parti du Minimum Spanning
Tree, i.e. u et v appartiennent au même ensemble
48         # Alors on ne veut pas ajouter cette arête au minimum
spanning-tree
49     e = e + 1 # Si u et v sont d'ensemble différent, on a atteint
un sommet de plus donc on incrémente
50     result.append([u, v, w]) # On ajoute la nouvelle arête au
résultat
51
52     return result
53
54 if __name__ == '__main__':
55     g = Graph(6)
56     g.add_edge(0, 1, 4)
57     g.add_edge(0, 2, 4)
58     g.add_edge(1, 2, 2)
59     g.add_edge(3, 4, 3)
60     g.add_edge(2, 5, 2)
61     g.add_edge(4, 5, 3)
62
63     result = kruskal(g)
64
65     for u, v, weight in result:
66         print(f"{u} - {v}: {weight}") # afficher le résultat

```

💡 Conseil

La sortie de l'algorithme est:

```

0 - 1 : 4
0 - 2 : 4
3 - 4 : 3
5 - 2 : 2
5 - 4 : 3

```

Il se lit comme une liste d'arêtes et de poids de l'arête correspondante.

Question 5: (🕒 10 minutes) Social Network Analysis : Papier

Les graphes peuvent être utilisés pour représenter une multitude de choses. L'une d'entre elles est la représentation de votre réseau d'amis. Imaginez que vous possédiez une liste de vos amis ainsi que des amis de vos amis (qui ne sont pas nécessairement vos amis). Cette liste peut-être représentée sous forme de graphe. Dans ce graphe:

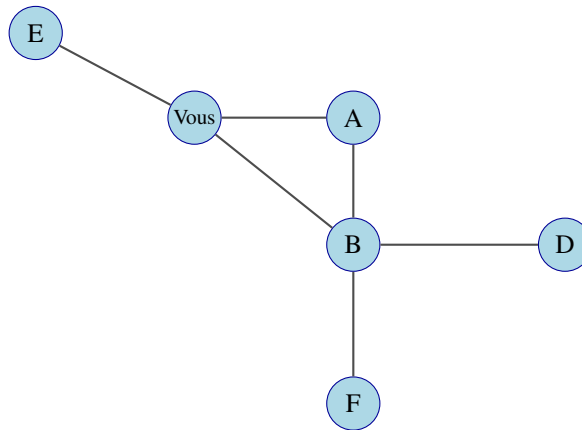
1. À quoi correspondent les arêtes et les sommets ?
2. Faut-il utiliser un graphe dirigé ?

Supposez que vous disposiez d'un graphe des relations sociales. Décrivez comment retrouver les éléments suivants :

1. Votre ami qui a le plus d'amis
2. Découvrir quels amis à vous se connaissent
3. Listez vos amis qui pourraient vous présenter quelqu'un que vous ne connaissez pas (ami d'ami qui n'est pas votre ami)

Vous voudriez désormais ajouter une nouvelle personne sur ce graphe, mais cette dernière n'est ni votre ami, ni l'ami d'un de vos amis. Quel sera son degré dans le graphe ?

Retrouvez les éléments (1) à (3) dans le graphe ci-dessous :



💡 Conseil

Réfléchissez en terme d'arêtes, de sommets, de degrés et de cycles.

>_ Solution

Dans le graphe des relations sociales, les arêtes correspondent à un lien d'amitié et les nœuds représentent les personnes. Il n'est pas nécessaire d'utiliser un graphe dirigé si l'on considère qu'une relation d'amitié est toujours réciproque.

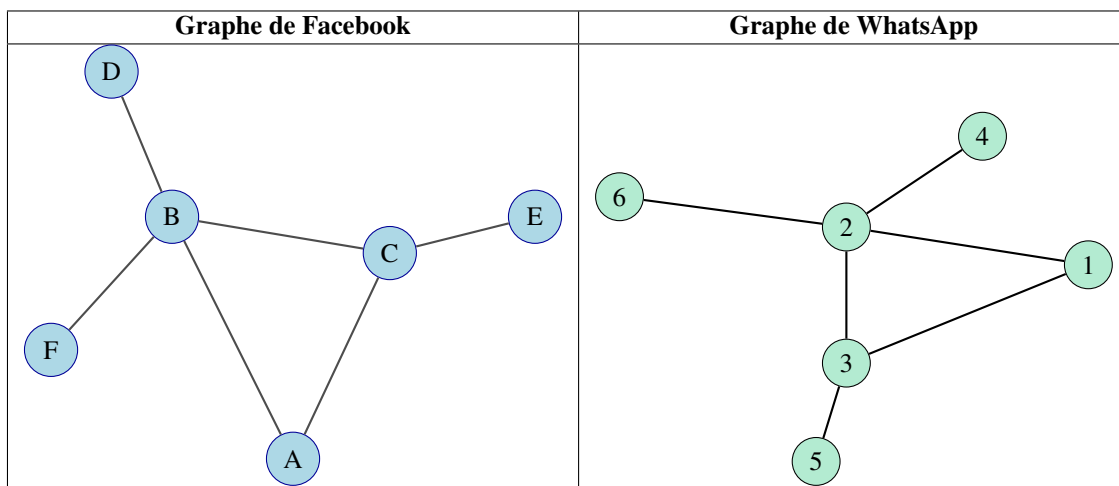
Pour trouver les éléments (1) à (3), il faut raisonner de la façon suivante :

1. Trouvez le sommet relié à vous qui a le degré le plus élevé. Sommet B dans le graphe.
2. Premièrement, les 2 personnes doivent être mes amis donc reliées à moi, mais de plus elles doivent être reliées entre elles. Par conséquent, cela correspond à un cycle dans le graphe. Il y a autant d'amis qui se connaissent que de cycles dans le graphe. Ami A et B dans le graphe.
3. Il ne doit pas y avoir d'arêtes me reliant avec l'ami de mon ami. Sommet B dans le graphe.

Si l'on ajoute une personne qui n'est ni un ami, ni l'ami d'un ami, alors aucune arête n'est reliée à ce sommet. Par conséquent, ce sommet a un degré 0.

Question 6: (🕒 5 minutes) Reconnaître des réseaux semblables

Supposons maintenant que vous travaillez chez Meta, qui a racheté Whatsapp, et que vous disposiez des deux graphes représentant respectivement Facebook et Whatsapp. Pouvez-vous déterminer visuellement à qui correspondent les individus de Facebook sur Whatsapp ?



💡 Conseil

Quelques conseils pour vous aider à résoudre cet exercice :

1. Identifiez les sommets des 2 graphes avec des caractéristiques semblables.
2. Il est possible que les sommets ne soient pas tous identifiables.

>_ Solution

1. B correspond à 2 (seul sommet de degré 4)
2. A correspond à 1 (seul sommet de degré 2 relié au sommet de degré 4)
3. C correspond à 3 (seul sommet de degré 3)
4. E correspond à 5 (seul sommet de degré 1 relié au sommet de degré 3)

Les sommets D et F et 4 et 6 ne peuvent pas être dissociés. On ne peut donc pas savoir qui correspond à qui.

Question 7: (🕒 5 minutes) Laquelle des affirmations suivantes est correcte ?

💡 Conseil

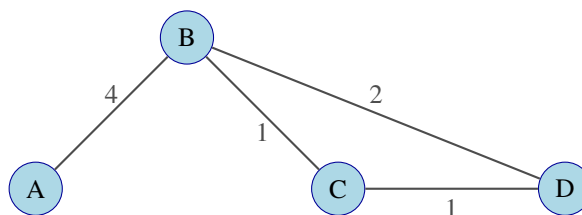
Un graphe complet est un graphe dans lequel tous les sommets sont adjacents (reliés entre eux).

1. L'algorithme du parcours en largeur (breadth-first search) ne se termine pas s'il existe un cycle dans le graphe.
2. Pour $|G| \gg \|G\|$ (nombre de sommets beaucoup plus grand que nombre d'arêtes), la matrice de contiguïté (adjacency matrix) occupe moins d'espace que la liste de contiguïté (adjacency list).
3. Dans un graphe complet, le nombre d'arêtes est $\frac{n(n-1)}{2}$ et le degré moyen d'un sommet est n où $n = |G|$.
4. Dans l'algorithme de Kruskal, l'instruction UNION est utilisée afin de déterminer si deux sommets sont déjà liés dans l'arbre couvrant minimal et de les relier si ce n'est pas le cas.

>_ Solution

1. Fausse : un sommet qui est déjà visité ne sera plus visité.
2. Fausse : la matrice de contiguïté a une complexité spatiale de $\Theta(V^2)$ tandis que la complexité spatiale de la liste de contiguïté est $\Theta(V + E)$.
3. Fausse : dans un graphe complet, chaque sommet est relié aux $n - 1$ autres sommets — la première partie de l'affirmation sur le nombre d'arêtes est quand même vraie.
4. Vraie : cela est exactement l'objectif de UNION.

Question 8: (🕒 10 minutes) Soit le graphe G donné ci-dessous, déterminer le nombre d'ensembles disjoints (disjoint sets) après les deux premiers appels de UNION dans l'algorithme de Kruskal.



Algorithm 1 MST-KRUSKAL(G, w)

```
1:  $A \leftarrow \emptyset$ 
2: for each vertex  $v \in G.V$  do
3:   MAKE-SET( $v$ )
4: end for
5: sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
6: for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight do
7:   if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ) then
8:      $A \leftarrow A \cup \{(u, v)\}$ 
9:     UNION( $u, v$ )
10:  end if
11: end for
12: return  $A$ 
```

>_ Solution

Les deux ensembles sont $\{A\}$ et $\{B, C, D\}$. Explication : Lors du premier tour de la boucle définie à ligne 6, l'arête BC (ou CD) est examinée. Par conséquent, B et C (ou C et D) sont reliés. Après la deuxième itération, les sommets B, C, et D sont reliés dans l'arbre couvrant minimal — ils appartiennent donc au même ensemble disjoint. A n'est pas encore relié après les deux premières itérations et le sera dans le quatrième tour de la boucle.